

Homemade Eval-Bar Breakthroughs: Cached Calibration, Integer WDL Tensorization, and Confidence-Adaptive Stability

Manus AI
Unchessed UCI Engine Research
September 2026

Abstract—This update reports four branch-local improvements to the engine-integrated homemade evaluation bar: material-indexed calibration caching, a precomputed integer WDL tensor, confidence-adaptive bounded smoothing, and provenance-safe UCI integration. The design is derived from the publicly documented Stockfish 19 evaluation-to-WDL semantics, but it explicitly does not claim exact SFNNv16 parity because the current source score is Unchessed’s hand-crafted evaluator. On the repository’s real seven-position Matetrack EPD corpus, the tensorized WDL projection achieved a measured 2.80x steady-state speedup over the exact logistic reference while producing an identical checksum and zero per-mille WDL difference. Six focused correctness tests passed, the workspace compiled, and the reviewer engine returned the new ‘evalbar’ command end to end.

I. MOTIVATION AND DESIGN CONTRACT

A useful evaluation bar must be fast enough to refresh frequently, accurate enough to remain monotone and interpretable, and stable enough not to oscillate under small evaluator noise. Stockfish 19 demonstrates that the displayed bar is a pipeline rather than a raw neural-network neuron: the engine applies material-dependent score conversion and a logistic WDL model [1–3]. The Unchessed experiment adopts these public semantics while preserving a strict provenance boundary: a proxy is labeled as a proxy, and its smoothed display value never feeds back into search.

The implementation is confined to ‘unchessed-core/src/eval_bar.rs’ and the requested ‘unchessed-evalbar’ experiment folder. The reviewer binary exposes ‘evalbar’ after a normal UCI ‘position’ command. The output contains White-centric centipawns, WDL per mille, expected score, smoothed score, and a provenance field.

II. BREAKTHROUGHS

A. Material-indexed calibration cache

Stockfish’s WDL model uses a cubic material-dependent calibration. The material index is an integer in the clamped range 17–78. Instead of evaluating the cubic polynomials on every refresh, the prototype constructs a 79-entry cache with ‘OnceLock’. This removes repeated floating-point polynomial work and makes the calibration deterministic after one-time initialization.

TABLE I
BREAKTHROUGH MECHANISMS AND SAFETY PROPERTIES

Mechanism	Speed/accuracy effect	Safety property
Material cache	Removes repeated cubic calibration	Same coefficients and material bins
Integer tensor	Removes runtime ‘exp’ in normal range	Exact reference cells, per-mille quantization
Adaptive smoothing	Faster settling when decisive; damped noise when equal	Raw score preserved; bounded display-only state
Provenance field	Prevents unsupported Stockfish claims	Explicit proxy label in CLI and UCI output

B. Integer WDL tensorization

The normal display range is clamped to $[-4000, 4000]$ centipawns. The prototype precomputes a tensor indexed by material class and display centipawn value. Each cell stores the rounded win/draw/loss tuple in per mille. Runtime conversion becomes an integer offset plus one array load, while scores outside the normal range use the exact fallback and mate sentinels remain discrete.

C. Confidence-adaptive bounded smoothing

The raw score remains available, but the visual state uses a bounded update. Confidence is derived from distance of the expected score from 0.5. Equal positions use stronger smoothing; decisive positions settle faster. Every update is capped at 180 cp before the adaptive blend. This is a presentation-only filter and cannot alter search decisions.

D. Provenance-safe integration

The UCI line reports ‘provenance=proxy’ because the source score is Unchessed HCE rather than the exact Stockfish 19 SFNNv16 network. This is an accuracy breakthrough in engineering practice: the implementation avoids false equivalence even when it uses Stockfish-derived calibration mathematics.

III. EXPERIMENTAL METHOD

The calibration accuracy test swept display scores from -1800 to +1800 cp in 37-cp increments and compared

TABLE II
MEASURED BENCHMARK RESULTS

Metric	Result
Real EPD positions	7
Rounds per method	10,000
Exact logistic time	4,964,388 ns
Tensor time	1,770,786 ns
Steady-state speedup	2.80x
Maximum WDL difference	0 per mille
Exact checksum	62,020,000
Tensor checksum	62,020,000
Focused tests	6 passed, 0 failed

the tensor path against an exact logistic implementation. It verified WDL symmetry, monotonicity, sum-to-1000 conservation, White-centric side normalization, rule-50 monotonic damping, and smoothing bounds.

For throughput, the experiment used the repository’s real ‘benchmarks/matetrack.epd’ corpus. It contains seven legal positions generated and verified by the project tooling. The benchmark ran 10,000 rounds over all seven positions after warming the tensor, so one-time construction was excluded from steady-state timing. This is a projection benchmark, not a claim about whole-engine nodes per second.

The result is concrete but appropriately narrow. The 2.80x value applies to the WDL projection on the sandbox CPU, not to the entire search. The identical checksum and zero measured difference show that the tensor preserves the tested reference outputs exactly for the benchmarked score/material combinations.

IV. END-TO-END ENGINE TEST

The release reviewer binary was built and fed the following UCI sequence:

```
uci
isready
position startpos
evalbar
position fen 4k3/8/8/8/8/8/3Q4/4K3 w - - 0 1
evalbar
quit
```

The engine returned ‘cp 14’, ‘wdl 32 957 11’, and ‘bar 0.510’ for the start position. For the legal queen-versus-king position it returned ‘cp 1164’, ‘wdl 1000 0 0’, and ‘bar 1.000’. Both lines included ‘provenance=proxy’. The workspace compiled successfully, and the six eval-bar tests passed after the legal-position and smoothing invariants were corrected.

V. LIMITATIONS AND NEXT RESEARCH STEPS

The current bar cannot rival Stockfish’s playing strength merely by changing presentation mathematics. Stockfish’s advantage comes primarily from its trained SFNNv16 evaluator, feature transformer, quantization, and search. This implementation rivals specific engineering properties of the bar projection: deterministic calibration, low refresh

cost, WDL conservation, monotonicity, and bounded visual stability. Exact SF19 parity would require loading and evaluating the matching network and reproducing its post-NNUE complexity, optimism, material, rule-50, and perspective pipeline [1–3].

The next scientifically meaningful step is a provenance-matched corpus containing Stockfish 19 ‘eval’/‘wdl’ outputs paired with Unchessed outputs. That corpus would support MAE, Brier score, calibration curves, and monotonicity audits without confusing a proxy’s presentation quality with engine playing Elo.

VI. CONCLUSION

The experiment adds three technical breakthroughs and one trust breakthrough. Material caching and tensorization make the WDL projection 2.80x faster on the real EPD benchmark with zero measured per-mille discrepancy. Confidence-adaptive smoothing improves visual stability without changing raw evaluation. Provenance-safe output prevents the homemade bar from claiming unsupported Stockfish equivalence. These results justify continued development while keeping the central limitation explicit: rivaling Stockfish’s full evaluation quality requires the trained evaluator, not only a better bar.

REFERENCES

- [1] Stockfish Team, “Stockfish 19,” Sep. 5, 2026. [Online]. Available: <https://stockfishchess.org/blog/2026/stockfish-19/>
- [2] official-stockfish, “Stockfish source at sf_19,” GitHub, commit edb0d9d. [Online]. Available: https://github.com/official-stockfish/Stockfish/tree/sf_19/src
- [3] official-stockfish, “WDL model,” GitHub. [Online]. Available: https://github.com/official-stockfish/WDL_model
- [4] official-stockfish, “UCI Protocol and Stockfish Commands,” Stockfish Documentation. [Online]. Available: <https://official-stockfish.github.io/docs/stockfish-wiki/UCI-Protocol-and-Stockfish-Commands.html>